TCSS 343 – Design and Analysis of Algorithms

Recurrences and Induction

Tight Bounds on Recurrences

- We have determined asymptotic bounds for these recurrences.

$$T(n) = \begin{cases} c & \text{if } n \leq 1 \\ T(n-1) + d & \text{if } n > 1 \end{cases}$$

$$U(n) = \begin{cases} c & \text{if } n \leq 1 \\ U\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + d & \text{if } n > 1 \end{cases}$$

$$V(n) = \begin{cases} c & \text{if } n \leq 1 \\ 2V\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + d & \text{if } n > 1 \end{cases}$$

- Using the recursion tree method we came up with these bounds.

$$T(n) \in \Theta(n)$$
$$U(n) \in \Theta(\log n)$$
$$V(n) \in \Theta(n)$$

- The recursion tree method is accurate but will only constitute a proof if we are extremely precise in our application (don't make off by one errors, don't ignore the floors, etc.).
  - Once we have a guess at a bound we can still prove our bound correct.

Tight Bounds on Recurrences

$$T(n) = \begin{cases} c & \text{if } n \leq 1 \\ T(n-1) + d & \text{if } n > 1 \end{cases}$$

Theorem: The recurrence $T(n) \in \Theta(n)$.

- We will split the theorem into two lemmas.
  - A lemma is an intermediary conclusion used to demonstrate the primary conclusion.

Lemma: The recurrence $T(n) \leq bn$ for all $n > n_0$.
  - This lemma establishes the upper bound.

Lemma: The recurrence $T(n) \geq an$ for all $n > n_0$.
  - This lemma establishes the lower bound.

- The theorem is the immediate result of the lemmas.

Upper Bounds on Recurrences

$$T(n) = \begin{cases} c & \text{if } n \leq 1 \\ T(n-1) + d & \text{if } n > 1 \end{cases}$$

Lemma: The recurrence $T(n) \leq bn$ for all $n > n_0$.

Proof: (by induction on n)

Base case ($n = 1$):

$$T(1) = c \leq b \cdot 1 = b$$

The base case is true so long as $b \geq c$.

Inductive Hypothesis: Assume that $T(k) \leq bk$ for all $k < n$.

Inductive Step ($n > 1$):

$$\begin{aligned} T(n) &= T(n-1) + d && \text{by definition of } T(n) \\ &\leq b(n-1) + d && \text{by Inductive Hypothesis} \\ &= bn - b + d \\ &\leq bn \end{aligned}$$

The last step is true only if $b \geq d$. Combining this constraint with the one above we select $b = \max(c, d)$.

By induction we have shown for all $n \geq 1$ that $T(n) \leq bn$. □

Lower Bounds on Recurrences

$$T(n) = \begin{cases} c & \text{if } n \leq 1 \\ T(n-1) + d & \text{if } n > 1 \end{cases}$$

Lemma: The recurrence $T(n) \geq an$ for all $n > n_0$.

Proof: (by induction on n)

Base case ($n = 1$):

$$T(1) = c \geq a \cdot 1 = a$$

The base case is true so long as $a \leq c$.

Inductive Hypothesis: Assume that $T(k) \geq ak$ for all $k < n$.

Inductive Step ($n > 1$):

$$\begin{aligned} T(n) &= T(n-1) + d && \text{by definition of } T(n) \\ &\geq a(n-1) + d && \text{by Inductive Hypothesis} \\ &= an - a + d \\ &\geq an \end{aligned}$$

The last step is true only if $a \leq d$. Combining this constraint with the one above we select $a = \min(c, d)$.

By induction we have shown for all $n \geq 1$ that $T(n) \geq an$. □

Inductive Proof Template

$$T(n) = \begin{cases} c & \text{if } n \leq 1 \\ T\big(f(n)\big) + g(n) & \text{if } n > 1 \end{cases}$$

Lemma: The recurrence $T(n) \leq bh(n)$ for all $n > n_0$.

Proof: (by induction on n)

Base case ($n = 1$):

Prove the base case and note the condition on b.

Inductive Hypothesis: Assume that $T(k) \leq bh(k)$ for all $k < n$.

Inductive Step ($n > 1$):

$$\begin{aligned} T(n) &= T\big(f(n)\big) + g(n) && \text{by definition of } T(n) \\ &\leq bh(f(n)) + g(n) && \text{by Inductive Hypothesis} \\ &\dots \\ &\leq bh(k) \end{aligned}$$

Note the condition on b.

Select b and n_0.

Conclude by induction that you have shown for all $n > 1$ that $T(n) \leq bh(n)$. □

Upper Bounds on Recurrences

$$U(n) = \begin{cases} c & \text{if } n \leq 1 \\ U\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + d & \text{if } n > 1 \end{cases}$$

Lemma: The recurrence $U(n) \leq b\log \mathrm{n}$ for all $n > n_0$.

Proof: (by induction on n)

Base case $(n = 1)$:

$$U(1) = c \leq b \cdot 0 = 0$$

The base case is true so long as $c \leq 0$.

But wait! By the definition of $O(\log n)$ we must have $c > 0$. So the base cases fails for $n = 1$.

Base case retry $(n = 2, 3)$:

The next value to try is 2 but if we want our base case to be complete we must also include 3.

Now notice that $U(2) \leq U(3)$ and $b \log 2 \leq b \log 3$ so if we prove

$$U(3) \leq b \log 2$$

we get

$$U(2) \leq U(3) \leq b \log 2 \leq b \log 3$$

and the base case would follow.

This is true when $b \log 2 = b \geq U(3) = U(1) + d = c + d$.

Upper Bounds on Recurrences

$$U(n) = \begin{cases} c & \text{if } n \leq 1 \\ U\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + d & \text{if } n > 1 \end{cases}$$

Lemma: The recurrence $U(n) \leq b\log \text{n}$ for all $n > n_0$.

Proof: (by induction on n continued)

Inductive Hypothesis: Assume that $U(k) \leq b \log k$ for all $k < n$.

Inductive Step ($n > 1$):

$$\begin{aligned} U(n) &= U\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + d && \text{by definition of } U(n) \\ &\leq b \log \left\lfloor \frac{n}{2} \right\rfloor + d && \text{by Inductive Hypothesis} \\ &\leq b \log \frac{n}{2} + d \\ &= b \log n - b \log 2 + d \\ &= b \log n - b + d \\ &\leq b \log n \end{aligned}$$

The last step is true only if $b \geq d$. Combining this constraint with the one above we select $b = c + d$.

By induction we have shown for all $n \geq 2$ that $U(n) \leq b \log n$. □

Lower Bounds on Recurrences

$$U(n) = \begin{cases} c & \text{if } n \leq 1 \\ U\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + d & \text{if } n > 1 \end{cases}$$

Lemma: The recurrence $U(n) \geq a \log n$ for all $n > n_0$.

Proof: (by induction on n)

Base case ($n = 1$):

$$U(1) = c \geq a \cdot 0 = 0$$

The base case is true no matter our choice of a.

Inductive Hypothesis: Assume that $U(k) \geq a \log k$ for all $k < n$.

Inductive Step ($n > 1$):

$$\begin{aligned} U(n) &= U\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + d && \text{by definition of } U(n) \\ &\geq a \log \left\lfloor \frac{n}{2} \right\rfloor + d && \text{by Inductive Hypothesis} \\ &\geq a \log \frac{n}{4} + d \\ &= a \log n - a \log 4 + d \\ &= a \log n - 2a + d \\ &\geq a \log n \end{aligned}$$

The last step is true only if $a \leq \frac{d}{2}$. We select $a = \frac{d}{2}$.

By induction we have shown for all $n \geq 1$ that $U(n) \geq a \log n$. □

Upper Bounds on Recurrences

$$V(n) = \begin{cases} c & \text{if } n \leq 1 \\ 2V\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + d & \text{if } n > 1 \end{cases}$$

Lemma: The recurrence $V(n) \leq bn$ for all $n > n_0$.

Proof: (by induction on n)

Base case ($n = 1$):

$$V(1) = c \leq b \cdot 1 = b$$

The base case is true so long as $b \geq c$.

Inductive Hypothesis: Assume that $V(k) \leq bk$ for all $k < n$.

Inductive Step ($n > 1$):

$$\begin{aligned} V(n) &= 2V\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + d && \text{by definition of } V(n) \\ &\leq 2b\left\lfloor \frac{n}{2} \right\rfloor + d && \text{by Inductive Hypothesis} \\ &\leq \frac{2bn}{2} + d \\ &= bn + d \\ &\leq bn \end{aligned}$$

The last step is true only if $d < 0$.

BUT $d > 0$! We are stuck. We cannot prove the upper bound this way. We need to start over.

Upper Bounds on Recurrences

$$V(n) = \begin{cases} c & \text{if } n \leq 1 \\ 2V\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + d & \text{if } n > 1 \end{cases}$$

Lemma: The recurrence $V(n) \leq bn - a$ for all $n > n_0$.

Proof: (by induction on n)

Base case ($n = 1$):

$$V(1) = c \leq b \cdot 1 - a = b - a$$

The base case is true so long as $b - a \geq c$.

Inductive Hypothesis: Assume that $V(k) \leq bk - a$ for all $k < n$.

Inductive Step ($n > 1$):

$$\begin{aligned} V(n) &= 2V\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + d && \text{by definition of } V(n) \\ &\leq 2b\left\lfloor \frac{n}{2} \right\rfloor - 2a + d && \text{by Inductive Hypothesis} \\ &\leq \frac{2bn}{2} - 2a + d \\ &= bn - 2a + d \\ &\leq bn - a \end{aligned}$$

The last step is true only if $a \geq d$. Combining this constraint with the one above we select $a = d$ and $b = c + d$.

By induction we have shown for all $n \geq 1$ that $V(n) \leq bn - a$.

Lower Bounds on Recurrences

$$V(n) = \begin{cases} c & \text{if } n \leq 1 \\ 2V\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + d & \text{if } n > 1 \end{cases}$$

Lemma: The recurrence $V(n) \geq an$ for all $n > n_0$.

Proof: (by induction on n)

Base case ($n = 1$):

$$V(1) = c \geq a \cdot 1 = a$$

The base case is true so long as $a \leq c$.

Inductive Hypothesis: Assume that $V(k) \geq ak$ for all $k < n$.

Inductive Step ($n > 1$):

$$\begin{aligned} V(n) &= 2V\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + d && \text{by definition of } V(n) \\ &\geq 2a\left\lfloor \frac{n}{2} \right\rfloor + d && \text{by Inductive Hypothesis} \\ &\geq 2a\left(\frac{n}{2} - 1\right) + d \\ &= an - 2a + d \\ &\geq an \end{aligned}$$

The last step is true only if $2a \leq d$. Combining this constraint with the one above we select $a = \min\left(c, \frac{d}{2}\right)$.

By induction we have shown for all $n \geq 1$ that $V(n) \geq an$. □

Exercises

$$T(n) = \begin{cases} c & \text{if } n \leq 1 \\ 2T(n-1) + d & \text{if } n > 1 \end{cases}$$

$$U(n) = \begin{cases} c & \text{if } n \leq 1 \\ U\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + dn & \text{if } n > 1 \end{cases}$$

$$V(n) = \begin{cases} c & \text{if } n \leq 1 \\ 3V\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + d & \text{if } n > 1 \end{cases}$$

- Use recursion trees to come up with a good guess for tight bounds on these functions and then use induction to prove your tight bound is correct.
- Here are the correct tight bounds in case you want to check your work.

$$T(n) \in \Theta(2^n)$$
$$U(n) \in \Theta(n)$$
$$V(n) \in \Theta\left(n^{\log_2 3}\right)$$

TCSS 343 – Design and Analysis of Algorithms

Merge Sort

Recursive Algorithms

- Divide and Conquer Design
  1. State the problem as formally as possible.
  2. Identify sub-problems.
  3. Find and prove correct a self-reduction from the problem to one or more of its sub-problems.
  4. Design a recursive algorithm from the structure of the self-reduction.
  5. Analyse the runtime of the algorithm using a recurrence.

- Let's see how John von Neumann applied this design technique in 1945 to develop the familiar Merge Sort.
  - Merge_sort
  - John_von_Neumann

- Example Problem: Given input of an unsorted list A output a sorted permutation of A.

```
Sort a List (SORT)
Input: A[1 … n] is a list of integers
Output: A′ a permutation of A such that
     A′[1] ≤ A′[2] ≤ … ≤ A′[n].
```



Sort Reductions

- Let $S(a, b)$ expresses the output for the SORT problem on input $A[a \ldots b]$.

Insertion Sort Reduction

$$S(a,b) = \begin{cases} A & \text{if } a = b \\ \text{insert}\big(A[a], S(a+1, b)\big) & \text{if } a < b \end{cases}$$

- Let $m = \left\lfloor \frac{a+b}{2} \right\rfloor$ be the midpoint between a and b.

Merge Sort Reduction

$$S(a,b) = \begin{cases} A & \text{if } a = b \\ \text{merge}\big(S(a,m), S(m+1, b)\big) & \text{if } a < b \end{cases}$$

Self-Reduction and Recursion

Insertion Sort Reduction

$$S(a,b) = \begin{cases} A & \text{if } a = b \\ \text{insert}\big(A[a], S(a+1,b)\big) & \text{if } a < b \end{cases}$$

```
InsertionSort(A[a … b])
        If a = b return A.
        Return Insert(A[a], InsertionSort(A[a + 1 … b])).
End InsertionSort
```

- The Insert operation takes dn operations.
- Let $T(n)$ express the worst case runtime of the insertion sort algorithm on an input list of size n.

$$T(n) = \begin{cases} c & \text{if } n \leq 1 \\ T(n-1) + dn & \text{if } n > 1 \end{cases}$$

Repeated Substitution

$$T(n) = \begin{cases} c & \text{if } n \leq 1 \\ T(n-1) + dn & \text{if } n > 1 \end{cases}$$

- A second way to guess a bound for a recurrence is by repeated substitution – effectively unwinding the recurrence.

$$\begin{aligned} T(n) &= T(n-1) + dn \\ &= T(n-2) + d(n-1) + dn \\ &= T(n-3) + d(n-2) + d(n-1) + dn \\ &= \cdots \\ &= T(n-k) + \textstyle\sum_{i=0}^{k-1} d(n-i) \end{aligned}$$

- We can unwind this recurrence $n-1$ times before $T\big(n-(n-1)\big) = T(1) = c$ and the recursion ends.
- So if we sub in $k = n-1$ we get

$$c + \sum_{i=0}^{n-2} d(n-i)$$

- We can reverse this sum.

$$c + \sum_{i=2}^{n} di = c + d\sum_{i=2}^{n} i = c + \frac{d(n+1)n}{2} - d \in \Theta(n^2)$$

Upper Bounds on Recurrences

$$T(n) = \begin{cases} c & \text{if } n \leq 1 \\ T(n-1) + dn & \text{if } n > 1 \end{cases}$$

Lemma: The recurrence $T(n) \leq bn^2$ for all $n > n_0$.

Proof: (by induction on n)

Base case ($n = 1$):

$$T(1) = c \leq b \cdot 1^2 = b$$

The base case is true so long as $b \geq c$.

Inductive Hypothesis: Assume that $T(k) \leq bk^2$ for all $k < n$.

Inductive Step ($n > 1$):

$$\begin{aligned} T(n) &= T(n-1) + dn && \text{by definition of } T(n) \\ &\leq b(n-1)^2 + dn && \text{by Inductive Hypothesis} \\ &= bn^2 - 2bn + b + d\text{n} \\ &\leq bn^2 \end{aligned}$$

The last step is true only if $2bn \geq dn + b$. Since $n > 1$ we know $bn > b$ so this condition holds if $bn \geq dn$.

We select $b = \max\{c, d\}$.

By induction we have shown for all $n \geq 1$ that $T(n) \leq bn^2$. □

- Prove the lower bound as an exercise.

Self-Reduction and Recursion

Merge Sort Reduction

$$S(a,b) = \begin{cases} A & \text{if } a = b \\ \text{merge}\big(S(a,m), S(m+1,b)\big) & \text{if } a < b \end{cases}$$

```
MergeSort(A[a ... b])
        If a = b return A.
        Let m = ⌊(a+b)/2⌋.
        Let l = MergeSort(A[a ... m]).
        Let r = MergeSort(A[m + 1 ... b]).
        Return Merge(l, r).
End MergeSort
```

- Let $T(n)$ express the worst case runtime of the merge sort algorithm on an input list of size n.

$$T(n) = \begin{cases} c & \text{if } n \leq 1 \\ T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + T\left(\left\lceil \frac{n}{2} \right\rceil\right) + dn & \text{if } n > 1 \end{cases}$$

Simplify the Recurrence

$$T(n) = \begin{cases} c & \text{if } n \leq 1 \\ T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + T\left(\left\lceil \frac{n}{2} \right\rceil\right) + dn & \text{if } n > 1 \end{cases}$$

- Notice that $T(n)$ is closely related to two other recurrences.

$$T_l(n) = \begin{cases} c & \text{if } n \leq 1 \\ 2T_l\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + dn & \text{if } n > 1 \end{cases}$$

$$T_u(n) = \begin{cases} c & \text{if } n \leq 1 \\ 2T_u\left(\left\lceil \frac{n}{2} \right\rceil\right) + dn & \text{if } n > 1 \end{cases}$$

- More precisely these recurrences are bounds on $T(n)$.

$$T_l(n) \leq T(n) \leq T_u(n)$$

- Since these recurrences are easier to use and we are looking for a tight bound on $T(n)$ we can focus on these recurrences instead.

Repeated Substitution

$$T(n) = \begin{cases} c & \text{if } n \leq 1 \\ 2T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + dn & \text{if } n > 1 \end{cases}$$

- Let's use repeated substitution to guess a tight bound. I will ignore the floor to make the math easier.

$$\begin{aligned} T(n) &= 2T\left(\tfrac{n}{2}\right) + dn \\ &= 2\left(2T\left(\tfrac{n}{4}\right) + \tfrac{dn}{2}\right) + dn \\ &= 4T\left(\tfrac{n}{4}\right) + dn + dn \\ &= 4\left(2T\left(\tfrac{n}{8}\right) + \tfrac{dn}{4}\right) + dn + dn \\ &= 8T\left(\tfrac{n}{8}\right) + dn + dn + dn \\ &= \cdots \\ &= 2^k T\left(\tfrac{n}{2^k}\right) + \textstyle\sum_{i=0}^{k-1} dn \end{aligned}$$

- We can unwind this recurrence $\log n$ times before $T\left(\frac{n}{2^{\log n}}\right) = T(1) = c$ and the recursion ends.
- So if we sub in $k = \log n$ we get

$$c2^{\log n} + \sum_{i=0}^{\log n - 1} dn$$

$$= cn + dn \sum_{i=0}^{\log n - 1} 1 = cn + dn \log n \in \Theta(n \log n)$$

Upper Bounds on Recurrences

$$T(n) = \begin{cases} c & \text{if } n \leq 1 \\ 2T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + dn & \text{if } n > 1 \end{cases}$$

Lemma: The recurrence $T(n) \leq bn \log n + a$ for all $n > n_0$.

Proof: (by induction on n)

Base case ($n = 1$):

$$T(1) = c \leq b \cdot 1 \cdot 0 + a = a$$

The base case is true so long as $a \geq c$.

Inductive Hypothesis: Assume that $T(k) \leq bk \log k + a$ for all $k < n$.

Inductive Step ($n > 1$):

$$\begin{aligned} T(n) &= 2T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + dn && \text{by definition of } T(n) \\ &\leq 2b \left\lfloor \frac{n}{2} \right\rfloor \log \left\lfloor \frac{n}{2} \right\rfloor + 2a + dn && \text{by Inductive Hypothesis} \\ &\leq 2b \frac{n}{2} \log \frac{n}{2} + 2a + d\text{n} \\ &= bn \log \frac{n}{2} + 2a + dn \\ &= bn \log n - bn + dn + 2a \\ &= bn \log n + a + (dn - bn + a) \\ &\leq bn \log n + a \end{aligned}$$

The last step is true only if $bn \geq dn + a$. If we select $a = c$ from the condition above then we can select $b = d + c$.

By induction we have shown for all $n \geq 1$ that $T(n) \leq bn \log n$. □

Lower Bounds on Recurrences

$$T(n) = \begin{cases} c & \text{if } n \leq 1 \\ 2T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + dn & \text{if } n > 1 \end{cases}$$

Lemma: The recurrence $T(n) \geq bn \log n$ for all $n > n_0$.

Proof: (by induction on n)

Base case ($n = 1$):

$$T(1) = c \geq b \cdot 1 \cdot 0 = 0$$

The base case is true for all b.

Inductive Hypothesis: Assume that $T(k) \geq bk \log k$ for all $k < n$.

Inductive Step ($n > 1$):

$$\begin{aligned} T(n) &= 2T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + dn && \text{by definition of } T(n) \\ &\geq 2b \left\lfloor \frac{n}{2} \right\rfloor \log \left\lfloor \frac{n}{2} \right\rfloor + dn && \text{by Inductive Hypothesis} \\ &\geq 2b \left(\frac{n}{2} - 1\right) \log \frac{n}{4} + d\text{n} \\ &= bn \log \frac{n}{4} - 2b \log \frac{n}{4} + dn \\ &= bn \log n - 2bn - 2b \log n + 4b + dn \\ &\geq bn \log n \end{aligned}$$

The last step is true only if $2bn + 2b \log n - 4b \leq 2bn + 2bn \leq 4bn \leq dn$. So we will select $b = \frac{d}{4}$.

By induction we have shown for all $n \geq 1$ that $T(n) \geq bn \log n$. □

TCSS 343 – Design and Analysis of Algorithms

Quicksort

Recursive Algorithms

- Divide and Conquer Design
  1. State the problem as formally as possible.
  2. Identify sub-problems.
  3. Find and prove correct a self-reduction from the problem to one or more of its sub-problems.
  4. Design a recursive algorithm from the structure of the self-reduction.
  5. Analyse the runtime of the algorithm using a recurrence.

- Let's see how Tony Hoare applied this design technique in 1960 to develop the familiar Quicksort and Quickselect algorithms.
  - Quicksort
  - Tony Hoare
    - Turing Award (1980)



- Example Problem: Given input of an unsorted list A output a sorted permutation of A.

  `Sort a List (SORT)`
  `Input:` $A[1 \dots n]$ `is a list of integers`
  `Output:` A' `a permutation of` A `such that` $A'[1] \leq A'[2] \leq \dots \leq A'[n]$.

Pivot

- The Quicksort and Quickselect algorithms both use a pivot to break the problem into sub-problems.

```
Pivot(A[a … b])
                    Let p = A[i] for some a ≤ i ≤ b.
                    Let P, L and R be empty lists.
                    For each element a of A:
                            If a = p add a to P.
                            If a < p add a to L.
                            If a > p add a to R.
                    Return (P, L, R).
End Pivot
```

- This function will take the elements of the list and place them in three groups:
  - The elements less than the pivot.
  - The elements greater than the pivot.
  - The elements equal to the pivot (we often assume all elements are unique for simplicity).
- Notice we haven't specified how to select the pivot. What would make a good pivot?
  - The median element is the best choice. The minimum and maximum elements are the worst choice.
- What is a good method for selecting a pivot?
  - Select the first element $A[a]$.
  - Select at random.
  - Select the median. How can this be done fast?

Pivot

- The Quicksort and Quickselect algorithms both use a pivot to break the problem into sub-problems.

```
Pivot(A[a … b])
        Let p = A[i] for some a ≤ i ≤ b.
        Let P, L and R be empty lists.
        For each element a of A:
                If a = p add a to P.
                If a < p add a to L.
                If a > p add a to R.
        Return (P, L, R).
End Pivot
```

- The runtime of this function is dominated by the for each loop in every case.
- Let $f(n)$ express the runtime of this function then

$$f(n) = c + \sum_{i=1}^{n} d$$

$$= c + dn$$

$$\in \Theta(n)$$

Reductions

- The pivot function returns three elements
  - The pivot, the left sub-problem, and the right sub-problem.

- Let $S(A)$ express the output of the SORT problem on input A.
  - We can write a reduction for SORT using the elements returned from pivot.

Quicksort Reduction

$$S(A) = \begin{cases} A & \text{if } |A| \leq 1 \\ S(\text{left}) + \text{pivot} + S(\text{right}) & \text{if } |A| > 1 \end{cases}$$

Recursion

Quicksort Reduction

$$S(A) = \begin{cases} A & \text{if } |A| \leq 1 \\ S(\text{left}) + \text{pivot} + S(\text{right}) & \text{if } |A| > 1 \end{cases}$$

```
Quicksort(A[a ... b])
    If a = b return A.
    Let (p, l, r) = Pivot(A[a ... b]).
    Let l = Quicksort(l).
    Let r = Quicksort(r).
    Return l + p + r.
End Quicksort
```

Runtime Analysis

- A difficulty in analysing Quicksort and Quickselect is that depending on how we select the pivot we don't really know the sizes of the left and right sub-problem.
  - If we magically select the median value then the size of each sub-problem is $\leq \left\lfloor \frac{n}{2} \right\rfloor$. This represents our best case.
  - If we select the minimum or maximum value then the size of one sub-problem is 0 and the other is $n - 1$. This represents our worst case.
- Pivot selection method:
  - Select the first value.
    - This can result in a best case or worst case selection.
  - Select a random value.
    - This can result in a best case or a worst case selection.
  - Select the median value.
    - This results in a best case but might take too long a time to be useful.

Quicksort

```
Quicksort(A[a ... b])
        If a = b return A.
        Let (p, l, r) = Pivot(A[a ... b]).
        Let l = Quicksort(l).
        Let r = Quicksort(r).
        Return l + p + r.
End Quicksort
```

Worst case:

$$T(n) = \begin{cases} c & \text{if } n = 1 \\ T(n-1) + dn & \text{if } n > 1 \end{cases}$$

- There is only one sub-problem consisting of all but the pivot.
- The cost of the pivot operation is dn.
- We proved earlier that $T(n) \in \Theta(n^2)$.

Quicksort

```
Quicksort(A[a … b])
        If a = b return A.
        Let (p, l, r) = Pivot(A[a … b]).
        Let l = Quicksort(l).
        Let r = Quicksort(r).
        Return l + p + r.
End Quicksort
```

Best case:

$$T(n) = \begin{cases} c & \text{if } n = 1 \\ 2T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + dn & \text{if } n > 1 \end{cases}$$

- There are two sub-problems both with size $\leq \left\lfloor \frac{n}{2} \right\rfloor$.
- The cost of the pivot operation is dn.
- We proved last lecture that $T(n) \in \Theta(n \log n)$.

TCSS 343 – Design and Analysis of Algorithms

Analysis of Quicksort

Pivot Selection

- Select the first element in the list as the pivot.
  - This selection method will pick good or bad pivots depending on the initial ordering of the elements in the list.
    - Sorted lists are bad cases.
    - Shuffled lists are good cases.
  - We can do an average case analysis of this algorithm to see how it performs on average over all $n!$ possible inputs.
- Select a random element in the list as the pivot.
  - This selection method will pick good and bad pivots depending on the random selection. The best or worst case is no longer dependent on the input.
    - Any list could have good or bad random choices.
  - We can do an expected worst case analysis of this algorithm.
- The average case and expected worst case analysis have similar reasoning so we will prove them at the same time.

Average Case and Expected Worst Case

- Assume without loss of generality that our list consists of the numbers $1 \ldots n$.
- Select the first number in the list as the pivot.
  - There are $n!$ possible inputs and $\frac{1}{n}$ of them have i as the first element.
- Select a random number in the list as the pivot.
  - There are n possible pivots selected uniformly with probability $\frac{1}{n}$.
- This yields the recurrence $T(n)$ expressing both the average case and expected worst case.

$$T(n) = \begin{cases} c & \text{if } n \leq 1 \\ dn + \frac{1}{n}\sum_{i=0}^{n-1} T(i) + T(n-i-1) & \text{if } n > 1 \end{cases}$$

- We can simplify the summation by rearranging the terms and dropping the $T(0)$ terms.

$$\frac{1}{n}\sum_{i=0}^{n-1} T(i) + T(n-i-1) = \frac{1}{n}\sum_{i=1}^{n-1} T(i) + \frac{1}{n}\sum_{i=0}^{n-1} T(n-i-1)$$

$$= \frac{1}{n}\sum_{i=1}^{n-1} T(i) + \frac{1}{n}\sum_{i=1}^{n-1} T(i)$$

$$= \frac{2}{n}\sum_{i=1}^{n-1} T(i)$$

Average Case and Expected Worst Case

$$T(n) = \begin{cases} c & \text{if } n \leq 1 \\ dn + \frac{2}{n}\sum_{i=1}^{n-1} T(i) & \text{if } n > 1 \end{cases}$$

Lemma: The recurrence $T(n) \leq bn \log n$ for all $n > n_0$.

Proof: (by induction on n)

Base case ($n = 2,3$):

$$T(2) \leq T(3) \leq \mathrm{b} \cdot 2\log 2 = 2b$$

The base case is true so long as $b \geq \frac{T(3)}{2}$.

Inductive Hypothesis: Assume that $T(k) \leq bk \log k$ for all $k < n$.

Inductive Step ($n > 3$):

$$\begin{aligned} T(n) &= dn + \tfrac{2}{n}\textstyle\sum_{i=1}^{n-1} T(i) && \text{by definition of } T(n) \\ &\leq dn + \tfrac{2}{n}\textstyle\sum_{i=1}^{n-1} bi \log i && \text{by Inductive Hypothesis} \\ &\leq dn + \frac{2bn^2 \log n - 2bn^2 + 2b}{n \ln 16} && \text{by Integration} \\ &= bn \log n - \frac{2}{\ln 16} bn + dn + \frac{2b}{n \ln 16} \\ &= bn \log n - \frac{2}{\ln 16} bn + dn + \frac{2bn}{4 \ln 16} && \text{if } n \geq 4 \\ &\leq bn \log n \end{aligned}$$

The last step is true only if $b \geq \frac{2 \ln 16}{3} d$. We select $b = \max\left\{\frac{2\ln 16}{3} d, \frac{T(3)}{2}\right\}$.

By induction we have shown for all $n \geq 2$ that $T(n) \leq bn \log n$. □

TCSS 343 – Design and Analysis of Algorithms

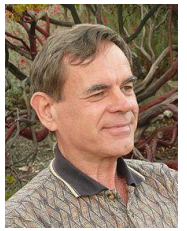
Median of Medians

Median of Medians

- In 1973 a team of famous computer scientists wanted to improve the worst case runtime of Quicksort and Quickselect by finding a procedure that would find a good pivot in $\Theta(n)$ operations.
  - Manuel_Blum
    - Turing Award (1995)
  - Robert_W._Floyd
    - Turing Award (1978)
  - Vaughan_Pratt
    - No Turing Award!
  - Ron_Rivest
    - Turing Award (2002)
  - Robert_Tarjan
    - Turing Award (1986)







- The algorithm they developed is also divide and conquer and is called the Median-of-Medians algorithm or the BFPRT algorithm.
  - Median_of_medians
  - The algorithm returns a pivot that in the worst case will divide the list into sub-problems of size $\frac{3n}{10}$ and $\frac{7n}{10}$.
  - This is a good enough pivot to ensure the worst case runtimes of Quicksort and Quickselect are $\Theta(n \log n)$.

Median of Medians

```
MedianOfMedians(A[1 ... n])
        If n < 5 return Select(A[1 ... n], n/2).
        Let M be an empty list.
        For i from 0 to n/5 − 1:
                Let m = Select(A[5i + 1 ... 5i + 5], 3).
                Add m to M.
        Return Quickselect(M[1 ... n/5], n/10).
End MedianOfMedians
```

- This recursive function calls a brute force solution to SELECT on lists of size ≤ 5.
- It is recursive because it calls Quickselect which is assumed to use this procedure to select pivots.
  - MedianOfMedians calls Quickselect and then Quickselect calls MedianOfMedians.
- We must argue two characteristics of this selection procedure.
  - The Median of Medians is greater than a linear number of elements.
  - The Quickselect algorithm when using Median of Medians is linear in the worst case.

Median of Medians

Lemma: The Median of Medians procedure returns a pivot that is greater than (and less than) at least $\frac{3n}{10}$ elements of the list.

Proof: The list M consists of $\frac{n}{5}$ medians of lists of size 5.

We know each of these medians is greater than at least 2 elements in the list and less than at least 2 elements in the list.

Quickselect will return the true median of the medians.

This true median is greater than (and less than) $\frac{\frac{n}{5}}{2} = \frac{n}{10}$ elements of M.

Since each of these elements is greater than (or less than) at least 2 other elements that means that the true median is greater than (and less than) at least $\frac{3n}{10}$ elements of the original list. □

- This means in the worst case in the Quickselect algorithm we will have to recurse on the larger of the two lists of size at most $\frac{7n}{10}$.
- When we use the Quickselect algorithm to select the median of medians the size of M is $\frac{n}{5}$.
- Then the worst case runtime of Quickselect using Median of Medians is

$$T(n) = \begin{cases} c & \text{if } n \leq 1 \\ T\left(\left\lfloor \frac{7n}{10} \right\rfloor\right) + T\left(\left\lfloor \frac{n}{5} \right\rfloor\right) + dn & \text{if } n > 1 \end{cases}$$

Median of Medians

$$T(n) = \begin{cases} c & \text{if } n \leq 1 \\ T\left(\left\lfloor \frac{7n}{10} \right\rfloor\right) + T\left(\left\lfloor \frac{n}{5} \right\rfloor\right) + dn & \text{if } n > 1 \end{cases}$$

Lemma: The recurrence $T(n) \leq bn$ for all $n > n_0$.

Proof: (by induction on n)

Base case ($n = 1$):

$$T(1) = c \leq b \cdot 1 = b$$

The base case is true so long as $b \geq c$.

Inductive Hypothesis: Assume that $T(k) \leq bk$ for all $k < n$.

Inductive Step ($n > 1$):

$$\begin{aligned} T(n) &= T\left(\left\lfloor \frac{7n}{10} \right\rfloor\right) + T\left(\left\lfloor \frac{n}{5} \right\rfloor\right) + dn && \text{by definition of } T(n) \\ &\leq b\left\lfloor \frac{7n}{10} \right\rfloor + b\left\lfloor \frac{n}{5} \right\rfloor + dn && \text{by Inductive Hypothesis} \\ &\leq b\frac{7n}{10} + b\frac{n}{5} + dn \\ &= \frac{9}{10}bn + dn \\ &\leq bn \end{aligned}$$

The last step is true only if $b \geq 10d$. We select $b = \max\{10d, c\}$.

By induction we have shown for all $n \geq 1$ that $T(n) \leq bn$. □

Conclusion

- Quicksort is a popular sorting procedure for a few reasons.
  - The worst case runtime of the simple algorithm is $\Theta(n^2)$.
    - Despite this the best case and average case runtime $\Theta(n \log n)$.
    - The expected worst case of the randomized version of Quicksort is also $\Theta(n \log n)$.
  - Using Median of Medians the worst case runtime of Quicksort improves to $\Theta(n \log n)$.
    - This method is not used in practice often because the simple algorithm and the randomized algorithm outperform it in real situations.
    - The two methods were hybridized in 1997 by Dave Musser to create a new sort called Introsort.
    - Introsort will attempt a randomized Quicksort unless the recursion is taking to long, in which case it switches to the Median of Medians solution.
    - Introsort is used in the Standard Template Library for C++.